[Pointer, double pointer]

int arr = 10;

int n;
cin>>n;
int arr[n];

10 Million

40MB

cin>>n;

int arr[100];

vector<int> arr(n)

400

arr[1000]

Stack | Heap

2000

40MB

arr: address

8 byte

RAM

RAM

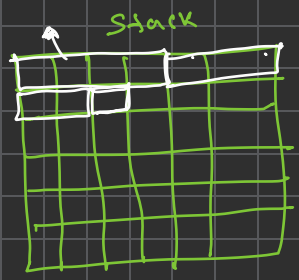
Code → Compile

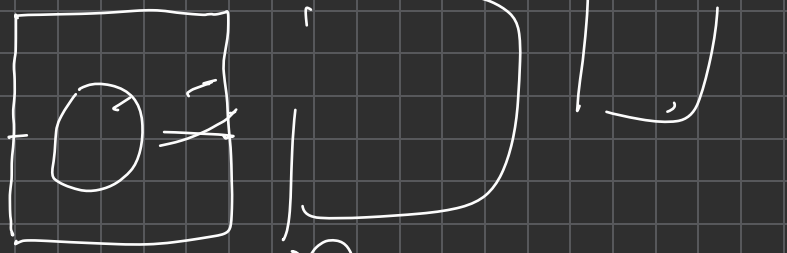
Run

① Syntax

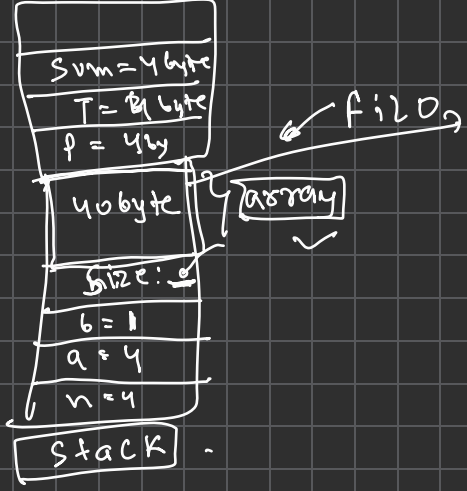
② Machine code

③ Memory requirement





vector<int> x(n)



int arr[40]

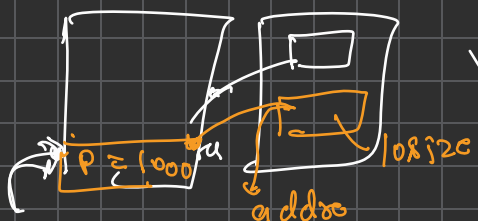
sum + =

40

int newarr[50]

→ 50

Heap



vector<int> arr(n)

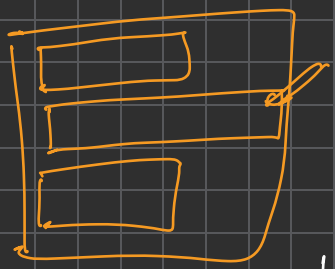
new
delete

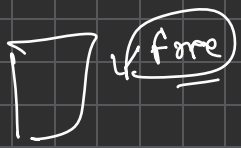
Tag di

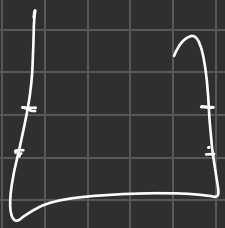
int a =

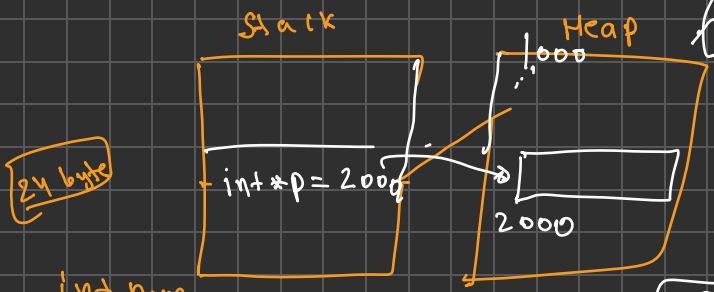
int arr[10] ;

int *p ⇒ new int[10]









Garbage Collector

Collector

Compile

main()

24 byte

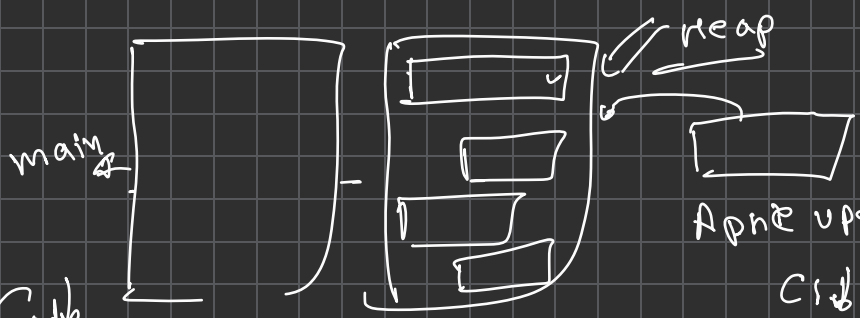
vector

```
int n,m
cin>>n,m;
int * p = new int[n];
int * arr = new int[m];
```

Function ⇒ stack

kitni

```
delete[] p;
delete[] arr;
```



Apne upar C

C++

GC

C#

GC

Code

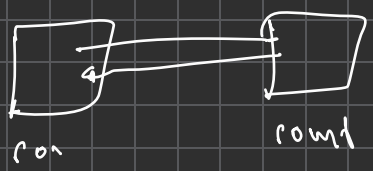
run

3-4 days

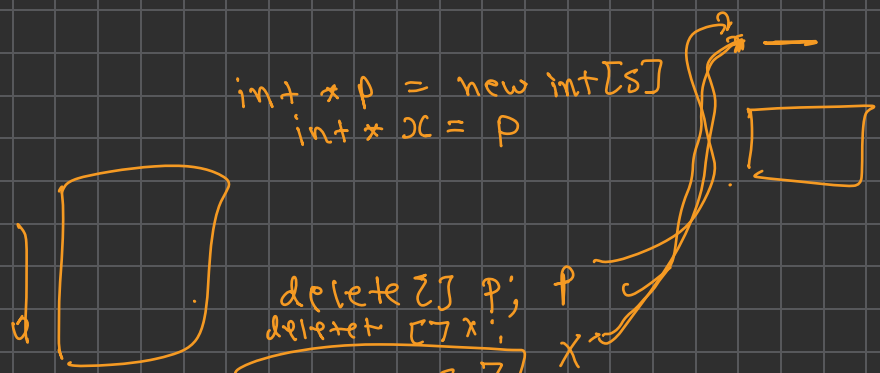
```
int * p = new int[4]
```

```
delete[] p;
```

dangling pointer

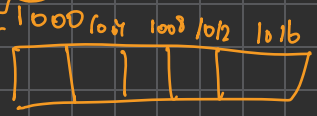



```
int * p = new int[5]
int * x = p
```




```
delete[] p;
delete[] x;
cout << x[0]
```

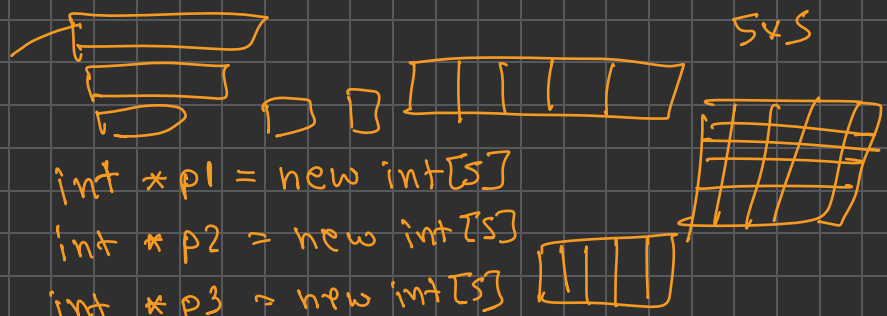
2 dimension array ?



p = 1000

```
int * p = new int[5]
```

2 dimension




```
int *p1 = new int[5]
int *p2 = new int[5]
int *p3 = new int[5]
int *p4 = new int[5]
int *p5 = new int[5]
```

```
cout << p[i][j];
```

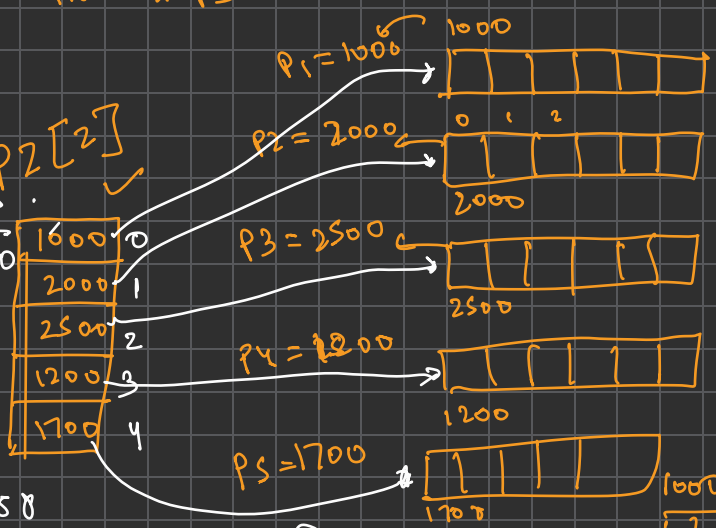
double point

arr=5000

5000

5x5

p2[2]

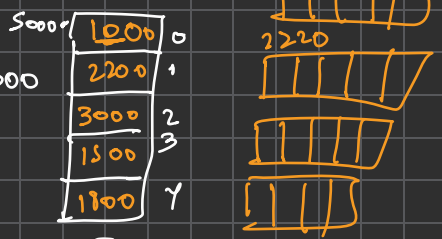


int ** arr

arr[2][2]

arr[i][j]

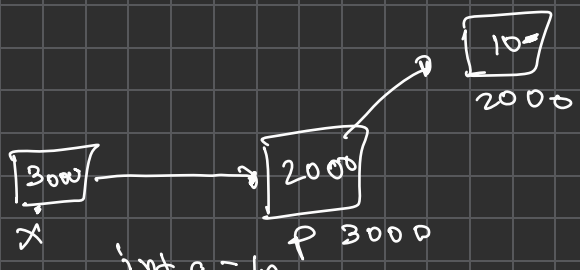
arr=5000



arr[1]

int ** arr = new int*[5]

```
for (int i=0 ; i<5 ; i++)
    arr[i] = new int[5];
```




```
int a = 10
int * p = &a;
int ** x = &p;
```

```
cout << **x;
```

x = 3000 ✓

*x = 2000

**x = 10 ✓